



Secure SDLC Assessment

The platform assessed against the Kosli Secure SDLC template: what is done, what is waiting, and why.

Prepared for	TAQA Cybersecurity
Prepared by	Mohammed Al-Jahdali
Assessed	main @ 01e7cd5, 4 Aug 2026
Platform	TAQA Knowledge Hub

INTERNAL USE

Contents

1. Scope	3
2. Status at a glance	3
3. Secure Build: 4 of 4 done	4
4. Secure Process: 2 partial, 2 pending	4
5. Security testing should follow integration	5
6. Secure Runtime: 2 done, 1 partial, 2 pending	6
7. Application-security findings, beyond the Kosli scope	6
8. Not yet started: Kosli supporting registers	8
9. Summary	8

1. Scope

The Knowledge Hub is a front-end application only.

- 42 files, about 2.2 MB of HTML, CSS and JavaScript
- No server, no backend, no database, no API, no user accounts
- No build step and no package manager. There is no `package.json` and no `node_modules`
- No real documents are stored. Document titles are reference metadata held in JavaScript, and the viewer shows a placeholder
- User state such as theme, bookmarks and read progress stays in the visitor's own browser and is never transmitted

This is not a disclaimer. It decides which controls can be assessed at all. About half the Runtime controls describe a running service with identity, logs and stored data, and the Hub has none of those yet. Those controls are not yet applicable rather than failed, and they become applicable when the platform moves to Azure.

2. Status at a glance

Group	Controls	Done	Partial	Pending
Secure Build	4	4	0	0
Secure Process	4	0	2	2
Secure Runtime	5	2	1	2
Total	13	6	3	4

Every pending item comes down to one of two causes. Either no IT owner has been assigned yet, or the platform is front-end only and has no runtime to secure.

3. Secure Build: 4 of 4 done

Control	Status	Evidence
Artifact Binary Provenance	Done	Every deployment traces to an exact commit SHA through the GitHub Actions logs. No compiled binary exists, since the deployed files are the source files.
Version Control	Done	Full Git history with no rewrites. Every change is attributable to an author and a commit.
Defined Toolchain	Done	Deliberately minimal: Git, then GitHub Actions, then Azure Static Web Apps. No compilers or bundlers to compromise.
Dependency Management	Done	There is no package manager and no dependency tree, so the largest software supply chain risk does not exist here. One vendored library, <code>qrcode.js</code> (MIT), was added to remove reliance on an external service.

Also completed under Build on 3 August 2026: all four third-party services were removed. Those were Google Analytics, Google Fonts, a Google Apps Script endpoint and an external QR service. All eleven pages were then loaded in an instrumented browser with all traffic recorded, and no requests reached any host other than our own.

4. Secure Process: 2 partial, 2 pending

Control	Status	Position and reason
Code Review	Pending	Branch protection is not enabled on <code>main</code> . The GitHub API reports <code>"protected": false</code> , so changes can reach the live site unreviewed. A review gate needs a second reviewer, and D&T has not yet nominated a technical reviewer (DT-02). Protection can be switched on immediately, and it becomes meaningful once a reviewer exists.
Quality Assurance	Partial	Every page is tested in a real browser before release, covering page errors, network calls and rendering. There is no automated test suite. Automated tests need real behaviour to assert against, such as login, document retrieval and upload, and none of that exists yet. Test automation should be built alongside the integration rather than before it.
Security Vulnerability Scanning	Partial	A full secret scan was run across every commit in the repository history and found no hardcoded credentials. A cross-site scripting vulnerability was found and fixed on 3 August 2026, covered in section 7. Automated SAST and DAST tooling and penetration testing are not yet in place, for the reasons in section 5.
Deployment Approvals	Pending	There is no approval gate, and deployment to the live site happens automatically on merge. This depends entirely on Code Review above. An approval gate without a nominated approver blocks work without adding assurance.

5. Security testing should follow integration

Penetration testing, DAST and malware scanning cannot produce useful results against the platform as it stands, because the things they test do not exist yet.

Test	Why it cannot run meaningfully today
Penetration test	Nothing to authenticate against, no session to hijack, no server to probe and no data to exfiltrate. A test today would examine a static file server rather than the system that will go live.
DAST	Dynamic scanners exercise running application logic. All logic currently runs in the visitor's own browser against fixed data.
Malware and antivirus scanning	Uploads are interface only. No file is stored or transmitted, so there is nothing to scan. This needs SharePoint or Azure Storage with Defender.
Authentication and session testing	There is no login, no session and no token.
Authorisation and access control testing	There are no roles and no permissions to test.
API security testing	There is no API.

The recommendation is to schedule the full security test cycle after Phase 1 (Entra ID sign-in) and Phase 2 (SharePoint document storage) are complete, so that it tests the real system. Running it now would spend budget on findings that do not map to the final architecture, and a full re-test would still be needed afterwards.

What was done instead suits a front-end at this stage: manual code review, a full-history secret scan, browser testing of all eleven pages, removal of external dependencies, and Content-Security-Policy enforcement.

6. Secure Runtime: 2 done, 1 partial, 2 pending

Control	Status	Position and reason
Change Records	Done	Every change is a commit, and every deployment is a logged GitHub Actions run tied to that commit. The record is complete and automatic, with no manual bookkeeping.
Deployment Controls	Partial	Deployment is automated, repeatable and traceable, with no manual copying or direct server access. HTTPS is enforced, and <code>Strict-Transport-Security</code> , <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> and <code>Referrer-Policy</code> are configured for the Azure deployment. Environment protection rules and a staging environment need the TAQA Azure subscription (DT-04).
Secrets Management	Done for current scope	No secrets exist in the platform, and none appear anywhere in the repository history. This was verified across all commits. The GitHub Actions secrets are referenced by name only, with values held in GitHub's encrypted store. Next step: the Entra ID client secret must live in Azure Static Web Apps configuration or Key Vault, never in code (DT-05).
Service Ownership	Pending	The Learning Center owns the platform on its own. There is no assigned IT owner, no support model and no escalation path. D&T has not yet assigned an IT coordinator (DT-01), which Ahmed Al Mubarak requested on 20 July 2026.
Workload Monitoring	Pending	There is no uptime monitoring, no server-side logging and no audit trail of who accessed which document. There is also no server to monitor, and a front-end produces no server-side logs. This becomes possible once the platform is hosted on TAQA Azure with SharePoint (DT-10, DT-19).

7. Application–security findings, beyond the Kosli scope

The Kosli template defines a process framework covering how software is built, reviewed, approved and deployed. It does not cover application-level security, and contains no controls for input validation, file handling or output encoding. This was checked against the template document itself.

An application-level review was therefore carried out in addition to the 13 Kosli controls. The findings below come from that review rather than from the framework, and all were identified and resolved on 3 August 2026.

Finding	Severity	Status
Cross-site scripting through a filename on the upload page. The selected filename was inserted into page HTML without escaping. A file named <code>.pdf</code> executed the script, confirmed by test rather than inspection. Impact today is limited to the user's own browser session because nothing is stored. After Phase 3 it would become stored XSS affecting every viewer of the document library.	Low now, high after integration	Fixed 3 August 2026. All user-controlled values are escaped. Re-tested with the same payload, which now renders as inert text with no execution. Existing behaviour was checked and is unaffected.
The upload page accepted executable file types (<code>.exe</code> , <code>.msi</code>). Once uploads reach SharePoint in Phase 3, this would have been a malware distribution path into a platform used by 5,000 staff.	Low now, high after integration	Fixed 3 August 2026. Executables are removed, and enforcement is a code-level allow-list rather than the input's <code>accept</code> attribute. That attribute is only a UI hint and is bypassed by drag and drop and by the picker's "All files" option. Verified through the bypass path: <code>.exe</code> and <code>.msi</code> are refused, double extensions such as <code>trick.pdf.exe</code> are caught, and the user is told why.
A function-name collision was silently suppressing upload warnings. The upload page defined its own toast function, and a later script overwrote it, so every toast on that page fired with mismatched arguments. A refused file would have failed silently.	Medium	Fixed 3 August 2026. The page-local function is renamed, and the rejection notice now displays correctly.
<code>.zip</code> archives remain permitted for legitimate multi-file documents. Archive contents cannot be inspected in the browser.	Low now, medium after integration	Open and accepted, to be mitigated at Phase 3. This needs server-side antivirus scanning through Defender for Storage or SharePoint once files are actually stored. Recorded here for the exception register.
<code>script-src</code> permits <code>'unsafe-inline'</code> , which is why the XSS above was able to execute.	Medium	Open and deferred. Removing it means relocating every inline script across roughly 13,000 lines. Better done during the Azure migration than as an isolated change.
The repository is public, so all code and document titles are readable. No credentials or real documents are exposed.	Low	Open, deliberate and time-boxed. GitHub requires a public repository for the free preview link. It should become private once TAQA hosting is live.

8. Not yet started: Kosli supporting registers

Item	Status	Note
Risk register	Not started	Should be created with D&T at handover
Exception register	Not started	Needed to record accepted risks such as <code>'unsafe-inline'</code>
Security training record	Not started	To follow TAQA's existing programme

9. Summary

Six of the 13 controls are done, and three defects found during this assessment were fixed the same day.

All four Secure Build controls are met, and the build posture is genuinely strong: no dependency tree, no secrets in history, no third-party calls, with full provenance and version control. Change Records and Secrets Management are met for the current scope. The three defects fixed were a confirmed cross-site scripting vulnerability, executable file types accepted by the upload page, and a function-name collision that was hiding upload warnings.

Three controls are pending because no IT owner has been assigned. Code Review, Deployment Approvals and Service Ownership all wait on D&T naming an IT coordinator and a technical reviewer (DT-01, DT-02).

Four are pending because the platform is front-end only. Workload Monitoring, full Quality Assurance, complete Vulnerability Scanning and full Deployment Controls need a running service with identity, storage and logs. The Hub has no backend yet, so these cannot be implemented or meaningfully tested.

One note on scope: the Kosli template covers the build and delivery process and has no application-security controls, so an application-level review was run alongside it. Section 7 lists what that review found.

For a front-end application at this stage the code-level security work is complete and can be evidenced. What remains is not neglected work. It becomes possible only once the platform has an identity provider, a server and stored data. Security testing should be scheduled after integration, and this assessment repeated at that point.

Mohammed Al-Jahdali. mohammed.jahdali@tq.com. +966 54 773 3744